

MySQL分库分表

参考文章

- <https://blog.csdn.net/ahuangqingfeng/article/details/124161107>
- https://blog.csdn.net/weixin_72610956/article/details/134461880

一、OA业务中用到的分库分表

通过shardingsphere完成数据库的分表

分片策略: AppCodeShardingAlgorithm

二、为什么需要分库分表&方式有哪些

文章: <https://blog.csdn.net/ahuangqingfeng/article/details/124161107>

1、TPS

TPS 是 **Transactions Per Second** 的缩写，意为“每秒事务处理量”，用于衡量系统在单位时间内处理事务的能力，常见于数据库、区块链和金融交易等领域。

2、分类概述

垂直拆分：

- 垂直拆分——数据库：一个大的系统库（电商库），按照业务拆分成各个小的模块，然后各自保存到对应的库中（订单库、会员信息库、商品库.....）
- 垂直拆分——数据表：将一个大表，根据某个字段拆分成各个小表，并通过这个拆分字段进行关联（如一个大的用户信息表，通过user_id字段，将用户账号信息拆成一个表，将用户信息拆成一个表）

目的：

- **优化性能**：减少单表数据量、I/O 操作，提高缓存效率。
- **提高可维护性**：模块化设计、降低耦合度、简化备份和恢复。
- **提高扩展性**：按业务扩展、支持微服务架构。
- **优化存储**：减少冗余、节省存储空间。
- **提高安全性**：数据隔离、减少数据泄露风险

垂直拆分特点：

- 每个库（表）的结构都是不一样的
- 每个库（表）的数据至少一列一样（即通过哪一个字段进行分库/表）
- 每个库（表）的并集即是全量数据

水平拆分：

- 水平拆分——数据库：将数据按某种规则分散到多个数据库中，每个数据库可能包含部分表或部分数据。
- 水平拆分——数据表：将一张表的数据按某种规则（如用户 ID 哈希、时间范围等）拆分到多个物理表中。

目的是解决单表数据量过大、性能瓶颈等问题，但同时也带来了新的挑战

水平拆分特点：

- 每个库（表）结构都是一样的
- 每个库（表）的数据都是不一样的，按照不同的策略分到不同的库（表）下
- 每个库（表）的并集就是全量数据

3.优缺点

- 垂直拆分：

优点：

- 1.拆分后业务清晰（专库专用按业务拆分）
- 2.数据维护简单，按业务不同，业务放到不同机器上

缺点：

- 1.如果单表的数据量大（某个业务的数据都存在一张表中），读写压力大
- 2.受某种业务决定（因为一个业务/模块相关的内容都存在一起），或者被限制，也就是说一个业务往往会影响到数据库的瓶颈（性能问题，如双十一抢购，短板效应，一个业务影响到其他业务的速率）
- 3.部分业务无法关联join，只能通过java程序接口去调用，提高了开发复杂度

关于部分业务无法关联join的解释

单机环境下，MySQL是支持跨库/表JOIN

在单机 MySQL 实例中，可以通过 `库名.表名` 的方式实现跨库或跨表的 `JOIN` 操作。例如：

```
SELECT *  
FROM db1.table1 t1  
JOIN db2.table2 t2 ON t1.id = t2.id;
```

这种方式在单机环境下是可行的，因为所有数据都存储在同一台服务器的同一个 MySQL 实例中

但在分布式环境下，就会有限制

在分布式环境下，数据可能分散在不同的 MySQL 实例或物理服务器上，此时跨库 `JOIN` 会面临以下问题：

（1）数据不在同一实例

- 在分布式系统中，不同库可能部署在不同的 MySQL 实例上，甚至位于不同的物理服务器。
- MySQL 本身不支持跨实例的 `JOIN` 操作，因为不同实例之间没有直接的通信机制。

（2）网络开销

- 即使通过程序逻辑实现跨库 JOIN，也需要从多个实例中分别查询数据，然后在应用程序中合并结果。
- 这种操作会带来大量的网络通信开销，导致性能下降。

(3) 事务一致性

- 在分布式环境下，跨库操作难以保证事务的 ACID 特性。
- MySQL 的分布式事务（如 XA 事务）性能较差，且配置复杂，通常不推荐使用。

(4) 性能问题

- 跨库 JOIN 需要对多个库的数据进行扫描和匹配，可能导致查询性能急剧下降。
- 如果数据量较大，查询可能会超时或失败。

(5) 扩展性差

- 分布式系统的设计初衷是为了分散负载和提高扩展性，而跨库 JOIN 会破坏这种设计，导致系统难以扩展。

具体解决方案：

在分布式环境下，通常不建议直接使用跨库 JOIN，而是采用以下替代方案：

(1) 数据冗余

- 将需要关联的数据冗余存储到同一个库或表中，避免跨库查询。
- 例如，将用户信息冗余到订单库中。

(2) 应用程序层 JOIN

- 在应用程序中分别查询不同库的数据，然后在内存中进行关联和计算。
- 例如，先查询用户库获取用户信息，再查询订单库获取订单信息，最后在程序中合并结果。

(3) 使用分布式数据库中间件

- 使用如 MyCat、ShardingSphere 等中间件，它们可以模拟跨库 JOIN，但实际是通过分步查询实现的。
- 这些中间件会隐藏底层复杂性，但对性能仍有影响。

(4) 数据同步

- 通过 ETL 工具或消息队列（如 Kafka）将不同库的数据同步到一个集中的库中，然后在该库中进行

为什么说 MySQL 不支持分布式环境下的跨库 JOIN？

- **功能限制：**MySQL 本身没有内置的分布式查询引擎，无法直接跨实例执行 JOIN。
- **性能问题：**即使通过 库名.表名 的方式实现跨库 JOIN，在分布式环境下性能通常无法接受。
- **事务和一致性：**分布式环境下，跨库操作难以保证事务一致性和数据完整性。

因此，在分布式系统中，通常认为 MySQL 不支持跨库 JOIN，而是通过其他方式（如应用程序层 JOIN 或数据冗余）来解决关联查询的需求。

- 水平拆分

优点：

- 1.单库/单表的数据保持在一定量（减少），有助于性能提高
- 2.提高了系统的稳定性和负载能力
- 3.拆分表的结构相同，程序改造较少。

缺点：

- 1.数据的扩容很有难度维护量大
- 2.拆分规则很难抽象出来
- 3.分片事务的一致性部分业务无法关联join，只能通过java程序接口去调用

自我总结：

- 表字段太多，库中表太多，可以根据业务进行垂直切分。
- 库/表中数据太多，可以进行水平切分。

三、分库分表带来的问题

1. 分布式事务
2. 跨库join查询
3. 分布式全局唯一id
4. 开发成本 对程序员要求高

四、分库分表的开源框架

分库分表是解决数据库性能瓶颈和扩展性的重要手段。为了实现分库分表，业界提供了多种开源框架，主要分为 **JDBC 直连层** 和 **Proxy 代理层** 两种类型：

- JDBC直连层：JDBC 直连层框架直接嵌入应用程序中，通过重写 JDBC 接口实现分库分表逻辑。应用程序直接与数据库交互，无需额外的代理服务。
- Proxy代理层：Proxy 代理层框架通过独立的代理服务实现分库分表逻辑，应用程序通过代理服务访问数据库，无需关心底层分库分表的细节。

一些常用的分库分表开源框架：

- jdbc 直连层：shardingsphere、tddl proxy
- 代理层：mycat, mysql-proxy (360)

(1) ShardingSphere

- **简介：**
 - ShardingSphere 是一套开源的分布式数据库中间件解决方案，前身是 Sharding-JDBC。
 - 它提供了分库分表、读写分离、分布式事务等功能。
- **核心组件：**
 - **Sharding-JDBC**：JDBC 直连层，嵌入应用程序中，实现分库分表逻辑。
 - **Sharding-Proxy**：数据库代理层，提供透明的分库分表服务。
 - **Sharding-Sidecar**：面向云原生的数据库治理方案（目前仍在开发中）。
- **特点：**
 - 支持多种分片策略（如按字段哈希、范围分片等）。
 - 支持读写分离和分布式事务。
 - 兼容 MySQL、PostgreSQL 等多种数据库。
 - 提供灵活的配置方式（YAML、Java API 等）。
- **适用场景：**
 - 需要将分库分表逻辑嵌入应用程序的场景。
 - 对性能要求较高的场景（直连模式无代理层开销）。

(2) TDDL (Taobao Distributed Data Layer)

- **简介：**
 - TDDL 是阿里巴巴开源的分布式数据库中间件，主要用于分库分表和读写分离。
 - 它是淘宝核心系统的重要组成部分，经过大规模生产环境的验证。
- **特点：**
 - 支持分库分表和读写分离。
 - 提供动态数据源管理功能。
 - 支持 SQL 解析和路由。
 - 与阿里巴巴的 Druid 连接池深度集成。
- **适用场景：**
 - 需要与阿里巴巴技术栈集成的场景。
 - 对分库分表和读写分离有较高要求的场景。

(3) MyCat

- **简介：**
 - MyCat 是一个开源的分布式数据库中间件，前身是阿里巴巴的 Cobar。
 - 它提供了分库分表、读写分离、数据分片等功能。
- **特点：**
 - 支持 MySQL、PostgreSQL 等多种数据库。
 - 提供透明的分库分表服务，应用程序无需修改代码。
 - 支持多种分片策略（如按字段哈希、范围分片等）。
 - 提供 SQL 解析、路由和优化功能。

- 支持高可用和负载均衡。
- **适用场景：**
 - 需要透明分库分表服务的场景。
 - 对应用程序侵入性要求较低的场景。

(4) MySQL-Proxy (360 Atlas)

- **简介：**
 - MySQL-Proxy 是一个基于 MySQL 协议的数据库代理工具，360 公司在其基础上开发了 Atlas。
 - Atlas 是 MySQL-Proxy 的增强版，提供了分库分表、读写分离、负载均衡等功能。
- **特点：**
 - 支持分库分表和读写分离。
 - 提供透明的数据库访问服务。
 - 支持 SQL 解析和路由。
 - 提供高可用和负载均衡功能。
- **适用场景：**
 - 需要透明分库分表服务的场景。
 - 对 MySQL 协议兼容性要求较高的场景。

JDBC 直连层 vs Proxy 代理层：

特性	JDBC 直连层 (如 ShardingSphere、TDDL)	Proxy 代理层 (如 MyCat、MySQL-Proxy)
性能	更高 (无代理层开销)	较低 (有代理层开销)
侵入性	较高 (需修改应用程序代码)	较低 (对应用程序透明)
部署复杂度	较低 (嵌入应用程序中)	较高 (需独立部署代理服务)
扩展性	较高 (与应用程序紧密集成)	较低 (受代理服务性能限制)
适用场景	对性能要求高、愿意接受代码侵入的场景	对应用程序侵入性要求低、需要透明服务的场景

五、影响数据库性能

- 数据量

MySQL单库数据量在5000万以内性能比较好,超过阈值后性能会随着数据量的增大而变弱。MySQL单表数据量是500w-1000w之间性能比较好,超过1000w性能也会下降。

- 磁盘

因为单个服务的磁盘空间是有限制的,如果并发压力下,所有的请求都访问同一个节点 (mysql实例),肯定会对磁盘IO造成非常大的影响。

- 数据库连接

数据库连接是非常稀少的资源,如果一个库里既有用户、商品、订单相关的数据,当海量用户同时操作时,数据库连接就很可能成为瓶颈。

为了提升性能,所以我们必须要解决上述几个问题,那就有必要引进**分库分表**。